Exploring Andrej Karpathy's Autoresearch: AI Agents Driving Autonomous ML Experimentation

In a recent X post, AI pioneer Andrej Karpathy unveiled his latest project, "autoresearch," a fascinating step toward agentic AI systems that conduct machine learning research independently. This self-contained repository demonstrates how AI agents can iterate on training code for a small language model, optimizing hyperparameters and architectures without human intervention. Drawing from Karpathy's background in training large neural networks at Tesla and OpenAI, autoresearch embodies a shift from manual tinkering to automated, agent-driven exploration. In this blog post, we'll dive into the technical details, unpack how it works, and highlight key insights for the future of AI research.

Project Overview

Autoresearch is built around a stripped-down version of nanochat, Karpathy's minimal LLM training framework, condensed into a single-GPU setup with about 630 lines of code. The core premise is simple yet powerful: humans refine a high-level prompt in a Markdown file (program.md), while an AI agent—powered by an external LLM like Claude or Codex—autonomously edits the training script (train.py) to experiment with improvements. The goal? Achieve the lowest possible validation bits per byte (val_bpb) in fixed 5-minute training runs, simulating rapid, iterative research cycles.

This setup transforms ML experimentation into an agentic loop, where the AI proposes code changes, runs experiments, evaluates results, and commits successful tweaks via Git. As Karpathy notes in his post, each dot in the accompanying visualization represents a complete LLM training run, with the agent accumulating commits as it discovers better configurations. The project is designed for accessibility, running on a single NVIDIA GPU like an H100, and encourages forking for adaptations to lower-end hardware.

Technical Implementation

At its heart, autoresearch uses PyTorch to train a simplified GPT-like model on datasets like TinyShakespeare or TinyStories. The system is divided into fixed and editable components to keep the agent focused:

- **Fixed Components**: prepare.py handles data preprocessing, including downloading datasets and training a byte-pair encoding (BPE) tokenizer with a default vocabulary size of 8192. It also provides dataloaders and evaluation utilities. This script remains untouched by the agent to ensure stability.

- **Editable Core**: train.py is the agent's playground. It defines the GPT model architecture (e.g., depth, embedding dimensions), optimizer (a blend of Muon and AdamW), and the training loop. Key hyperparameters the agent can tweak include:

    - DEPTH: Model layers (default 8, reducible to 4 for efficiency).

    - vocab_size: Adjustable down to 256 for byte-level fallback.

    - MAX_SEQ_LEN: Sequence length, often lowered on constrained hardware.

    - DEVICE_BATCH_SIZE and TOTAL_BATCH_SIZE: Power-of-2 values, e.g., scaling down to 16K tokens for smaller GPUs.

    - WINDOW_PATTERN: Attention mechanisms, like "SSSL" (default) or simplified "L" for performance.

    - Optimizer settings: Learning rates, warmup steps (e.g., from 0.5 to 4.7 in observed improvements), and cooldowns.